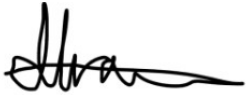


Project EPR400/402 First semester report	27 July 2026	Note:

Student to complete this section				
Braune	D.R.	Mr.	23539195	
Handwriting Robot			Study leader: Prof. H.C. Myburgh	
BEng Computer Engineering	Project number:	HCM4	Revision number:	1
Student declaration: I understand what plagiarism is and that I have to complete my project on my own. I am fully aware of the University's policy in this regard. I have not used another student's past or present written work to submit as my own. I declare that this report is my own original work. Where other people's work has been used (either from a printed source, internet or any other source), this has been properly acknowledged and referenced in accordance with departmental requirements.	 27/7/2026 Student signature Date			

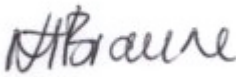
Declaration by language editor	
I have been allowed adequate time to read this document carefully and to make corrections where necessary (date received indicated below). To the best of my knowledge, correct formatting, spelling and grammar are used throughout the document.	
	27/7/2026
M.T. Braune	Date

Table of contents

1. Literature study	1
2. Work breakdown and first semester progress	3
3. Project plan (second semester)	6
4. References	8

1. Literature study

i) Existing Concepts

A need exists to be able to decode the writer's style and written text from an image:

The ability of a computer or system to identify written text after being trained on large amounts of data is known as HWR (Handwriting Recognition) or HTR (Handwritten Text Recognition) [1].

One type of handwritten text recognition is offline recognition, which means the data being identified by the system is static and captured post-completion of the page of handwritten text. This is ideal for identifying the text from a fixed image of handwritten content by utilizing image processing techniques and algorithms to identify the text that was written [3]. The other type is online recognition, which is a more dynamic process that identifies text during the creation of the handwriting, adding the variable of time to the classification of text.

Every stage of the process to achieve accurate handwritten text recognition requires a deep understanding of the different techniques available:

Datasets and resolution: Multiple datasets have typically been used for training handwritten text recognition modules, for example, the EMNIST, IAM and HEIC datasets [5]. The EMNIST (Extended Modified National Institute of Standards and Technology) dataset consists of approximately 814,255 handwritten characters and numbers, each being 28 by 28 pixels in size [1]. There is only a single, disconnected character per image, which is used in many existing HTR models, but these models cannot effectively decipher connected handwritten words, like cursive words. The IAM handwriting dataset provides much more variety, as it consists of 5,685 complete sentences, with a total of 115,320 words, gathered from 627 different authors, consisting of 26 letters and 27 punctuation marks [3]. The images from the IAM dataset were scanned at 300 dpi with 8 bits per pixel in grayscale. The dataset used influences the accuracy for the classification of text with higher variation, as well as defines the resolution of the camera required to capture the handwritten text to classify.

Pre-processing: The pre-processing of handwriting includes multiple techniques to be performed on the handwritten text before being passed into the classification and feature recognition algorithms. This is used to account for dimensional adjustments, removal of inconsistencies, and normalization of the data, to make the images more appropriate for text separation and classification [6]. Normalization is used to increase the accuracy of HTR models by reducing the effect that unwanted features of the data could have on the performance of the model. There are multiple different normalization techniques such as min-max scaling, z-score normalization, and log-scaling. Encoding techniques are also used in HTR model pre-processing to get rid of unintentional biases [1].

Text Separation: Text separation is the process of breaking up a page of handwritten text into smaller segments or blocks that can be used in feature extraction and neural

network classifiers. There are two methods that existing models employ to separate text: either individual character separation or word separation. Both methods involve breaking down paragraphs of handwritten text into smaller segments. The accuracy of the extraction of smaller segments heavily influences the effectiveness of the classifier. Some existing techniques for this are classic dissection techniques, recognition-based segmentation, hysteresis thresholding, and bounding box techniques [1].

To achieve either word or character separation, the page must still be broken up into individual lines and then into separate words. Line separation is sometimes done by converting images to grayscale, obtaining inverted binary images, and then drawing bounding boxes over each line [6]. Character separation just goes one step further and splits the word into individual characters.

Types of Neural Networks: Many different types of neural network models have been implemented in previous papers, including ANNs (artificial neural networks), CNNs (convolutional neural networks), RNNs (recurrent neural networks), and various combinations of them to achieve the best accuracy.

ANNs are created from the standard neural network architecture consisting of multiple neurons, weights, hidden layers, biases, etc., and can accurately perform pattern recognition for handwritten text. CNNs are built on the architecture of ANNs but also facilitate feature extraction and identifying smaller, more important information in text. This leads to a reduction of necessary learnable parameters and streamlines model training [3]. RNNs contain the same fundamental structure as CNNs but also have the capability to consider sequential and dependent data. The ability to capture time-dependent patterns is useful for deciphering various handwritten fonts, and because RNNs can consider dependencies in the data, this can improve the accuracy of handwritten text recognition when considering common linguistic dependencies in language [3].

A combination of both CNNs and RNNs can significantly enhance feature extraction by using the CNN's pattern recognition and ability to identify diverse handwriting styles, and the RNN's ability to use textual dependencies for improved contextual understanding [4].

Existing Models: Two baseline models have been primarily considered in existing research papers: the LeNet-5 CNN model and the ResNet-34 CNN model. LeNet-5 is the most common of the two and was created by Yann LeCun in 1988. It is a CNN structure that consists of 8 layers, including 3 convolutional layers, 2 pooling layers, and lastly, 2 fully connected layers. A fully connected layer refers to a portion of a typical ANN model, while the convolutional layers and pooling layers will be discussed later in this literature study. The input size to the model is 32 by 32 pixels, and the model uses a hyperbolic tangent activation function in the fully connected layers and a softmax activation function in the output layer. This model is popular for use in object detection and image classification [1].

ResNet-34 is also a popular CNN model known for its residual architecture. This model uses a concept known as transfer learning to retain its existing weights, which forms the foundation of the feature extraction in this model and allows for fast and efficient training from limited datasets [3].

Feature extraction: The convolutional layers of a CNN are used to perform feature extraction, which means they can capture subtle patterns in different areas of a segment of text by sliding a convolutional kernel over the character or word image. Translation invariance is created by sliding the convolution kernel, which allows consistent patterns to be extracted from images [3]. An N by N input layer is convolved with an f by f kernel and returns an $(N - f + 1)$ by $(N - f + 1)$ output. Feature maps are created by applying convolutions to different sections of the image being classified. By identifying common features, the kernel-sliding convolutional technique can produce more accurate classification. Each extracted feature is multiplied with a corresponding kernel value to produce a weighting, and then a bias and activation function are applied until an output feature map is obtained [7].

An extended version of the standard CNN is the deep CNN, which refers to a CNN that can extract higher-complexity features like handwritten strokes. The problem is that a larger, deeper CNN creates a ‘vanishing gradient’ problem where there are so many layers that during backpropagation, some layers never get their weightings changed. Thus, there is a limit to the number of layers a standard CNN can have, where adding more layers yields no further benefit. This is why a residual network is used to implement a deep CNN by using residual blocks and skip connections across various layers so that all layers will still update their weightings during backpropagation regardless of the number of layers [3].

After a singular round of feature selection, many parameters are still required to classify text with these high-level features. Max-pooling is used to downsample these feature maps until only the essential features remain, minimizing the number of parameters needed to classify handwritten characters and preventing overfitting [8].

A need exists to reproduce text on paper accurately and fast in any writer’s style:

There are various existing text reproduction methods and linear actuation techniques which can influence the accuracy and speed of text reproduction.

Method of reproduction: There are three main methods to reproduce handwritten text on paper: Cartesian systems, serial-chain robots, and parallel-chain robots. A Cartesian system consists of a gantry with $x - y$ movement. It has restricted movement in only two directions using linear actuation. Serial-chain robots are the most common in industrial settings for automated movement. They consist of multiple linkages chained together, one component connected to the next in a single chain, like a typical robot arm. The third method is parallel-chain robots, which consist of multiple serial-chain linkages that can precisely control automated movement [2].

Pen mapping techniques: The method of pen trajectory mapping for text reproduction influences the accuracy and speed of reproduction. A popular algorithm for pen mapping is the Douglas-Peucker algorithm, which takes a sequence of points and creates an estimation of the sequence by drawing points from the beginning to the end. The

algorithm produces one of the highest-accuracy estimations when compared to similar algorithms, yet it is not completely optimal [2].

Linear actuation types: There are multiple methods of linear actuation that could be considered for a Cartesian plotting system. Some popular examples of linear actuation methods are electric, pneumatic, belt-drive, rack-and-pinion, and ball-screw systems. Electric and pneumatic methods are often used for industrial cases, but belt-drive, rack-and-pinion, and ball-screw linear actuators have been used in multiple handwriting reproduction systems due to their durability, reliability, and cost-effectiveness [9].

ii) Intended Contribution and Discussion

The existing literature that has been discussed forms the foundation for the handwriting recognition and recreation system that must be created. Extensive knowledge of all the types of neural networks and techniques that must be used is needed to meet the accuracy and speed requirements for the classifier of this project. A deep understanding of the methods of text reproduction and pen mapping is also needed to ensure the accuracy and speed requirements for the reproduction part of the project. A pitfall of most research done in these areas is that it only focuses on individual elements of the full system and also makes use of many online libraries, for example, TensorFlow and OpenCV [1]. Therefore, the main contribution made to existing research will be the full integration of the system, including dataset creation, text classification, and text reproduction from first principles. The system will also focus on classifying various authors of the text as well as classifying the text that was written.

2. Work breakdown and first semester progress

Table 1. Project work breakdown and current completion

Task	Progress (% complete)	Brief description
Finish project proposal	100%	Revision 1 was approved.
Research neural networks in depth	100%	All aspects of neural networks have been researched and documented.
Full conceptual understanding of the system	100%	A full understanding of the overall flow and the method to achieve the requirements has been achieved.
CAD design of gantry	100%	A full gantry has been modelled from first principles using Autodesk Fusion 360.
Obtaining parts and 3D printing components	50%	Local parts have been acquired and printed, but some parts including the camera module have been ordered but must still be delivered.
Neural network design and implementation	0%	The classifier and deep learning model must be designed (preprocessing techniques and layers etc.) and code must be written to be used in the single board computer.
Simulation and training of classifier	0%	The classifier must be run on a PC with available datasets to verify accuracy of various model approaches to determine the best one.
Design and print PCB/ Solder on Veroboard	0%	The circuitry for the gantry and power supply must be designed using KiCad and printed locally on PCB, or properly soldered on Veroboard.
Construction and testing of gantry	0%	The gantry must be fully assembled and tested for speed and accuracy with simple pen mapping commands.
Integration with camera, SBC and gantry	0%	The full system must be assembled ensuring the correct power is supplied to each subsystem.
Validate requirements through testing	0%	The system must be fully tested in the specified field conditions to check if requirements are met, otherwise changes must be made.
Write final report	0%	The final design report must be written and submitted before the deadline

3. Project plan (second semester)

Start Week		Jul 20, 2026																				
Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	Notes	
Starting	Jul 20	Jul 27	Aug 3	Aug 10	Aug 17	Aug 24	Aug 31	Sept 7	Sept 14	Sept 21	Sept 28	Oct 5	Oct 12	Oct 19	Oct 26	Nov 2	Nov 9	Nov 16	Nov 23	Nov 30		
Phase One	Obtain parts and 3D prints																				1. Phase one refers to the design and obtaining of parts, prints and PCBs	
	Neural network design and implementation																					
			Design and place PCB order																		2. Phase two refers to the construction, integration and validation of the system	
Phase Two			Simulation and training of classifier																			
							Construction and testing of gantry															
									Full integration of system													
												Validate requirements through testing										3. Phase three refers to reports and demonstration
Phase Three												Write final report										
															Report submission and demo							

Figure 1. Second semester project plan, split into 3 stages of work to be completed (All tasks have been explained in the previous section).

4. References

1. A. Chen, "Handwritten text classification based on convolutional neural network," *Highlights Sci., Eng. Technol.*, vol. 34, pp. 39–44, 2023. [Online]. Available: https://www.researchgate.net/publication/369471914_Handwritten_Text_Classification_on_Based_on_Convolutional_Neural_Network. [Accessed: Jul. 15, 2026].
2. O. Teikari, "Reproducing handwriting with position-controlled robots," M.S. thesis, Dept. Signal Process., Tampere Univ. Technol., Tampere, Finland, 2016. [Online]. Available: <https://trepo.tuni.fi/bitstream/handle/123456789/24419/Teikari.pdf>. [Accessed: Jul. 27, 2026].
3. J. Ma, "A study on a hybrid CNN-RNN model for handwritten recognition based on deep learning," in *Proc. 1st Int. Conf. Data Anal. Mach. Learn. (DAML)*, 2023, pp. 275–280. [Online]. Available: <https://www.scitepress.org/Papers/2023/128010/128010.pdf>. [Accessed: Jul. 27, 2026].
4. J. P. Kumar and H. D. Dharshan, "Handwritten text recognition using deep learning: A CNN–LSTM approach," *Int. J. Intell. Syst. Appl. Eng.*, vol. 12, no. 1, pp. 45–52, 2024. [Online]. Available: <https://www.ijisae.org/index.php/IJISAE/article/view/7443/6436>. [Accessed: Jul. 27, 2026].
5. F. A. Mamun, S. A. H. Chowdhury, J. E. Giti, and H. Sarker, "Classification of non-native handwritten characters using convolutional neural network," *arXiv preprint arXiv:2406.04511*, 2024. [Online]. Available: <https://arxiv.org/pdf/2406.04511>. [Accessed: Jul. 27, 2026].
6. G. R. Hemanth, M. Jayasree, S. K. Venii, P. Akshaya, and R. Saranya, "CNN-RNN based handwritten text recognition," *ICTACT J. Soft Comput.*, vol. 12, no. 1, pp. 2457–2463, Jan. 2022. [Online]. Available: https://ictactjournals.in/paper/IJSC_Vol_12_Iss_1_Paper_1_2457_2463.pdf. [Accessed: Jul. 27, 2026].
7. A. A. Yahya, J. Tan, and M. Hu, "A novel handwritten digit classification system based on convolutional neural network approach," *Sensors*, vol. 21, no. 18, art. no. 6273, Sep. 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/18/6273>. [Accessed: Jul. 27, 2026].
8. S. Ji and Y. Zeng, "Handwritten character recognition based on CNN," in *Proc. 3rd Int. Conf. Artif. Intell. Adv. Manuf. (ALAM)*, Oct. 2021, pp. 2874–2880. [Online]. Available: <https://dl.acm.org/doi/fullHtml/10.1145/3495018.3501199>. [Accessed: Jul. 27, 2026].
9. IQS Directory, "Types of linear actuators," *IQS Directory Technical Library*, 2024. [Online]. Available: <https://www.iqsdirectory.com/articles/linear-actuator/types-of-linear-actuators.html>. [Accessed: Jul. 27, 2026].